inode cache, and is expected to sync that with the inode in the physical hardware-space file system. That is achieved by calling the appropriately modified *sfs_update()* (defined in *real_sfs_ops.c*, adapted from the earlier *browse_real_sfs* application). The key parameter changes are passing the inode number instead of the filename, and the actual timestamp instead of the flag for its update status. Accordingly, *sfs_lookup()* is being replaced by *sfs_get_file_entry()*, and additionally, the data blocks are now being freed (using *sfs_put_data_block()*), if the file size has reduced. Note that *sfs_put_data_block()* (defined in *real_sfs_ops.c*) is a transformation of the *put_data_block()* from the *browse_real_sfs* application. And finally, underlying write is being achieved using *write_to_real_sfs()*, a function added in *real_sfs_ops.c*, which is very similar to *read_from_real_sfs()* (already there in *real_sfs_ops.c*) —except for the direction reversal of the data transfer, and marking the buffer dirty to be synced up with the physical content. *sfs_write_inode()* and *sfs_update()* are shown in the code below. For the others, check out the file *real_sfs_ops.c*.

```
#if (LINUX_VERSION_CODE < KERNEL_VERSION(2,6,34))
static int sfs_write_inode(struct inode *inode, int do_sync)
#else
static int sfs_write_inode(struct inode *inode, struct writeback_
control *wbc)
#endif
{
    sfs_info_t *info = (sfs_info_t *)(inode->i_sb->s_fs_info);
    int size, timestamp, perms;

    if (!(S_ISREG(inode->i_mode))) // Real SFS deals only with
regular files
        return 0;

    size = i_size_read(inode);
    timestamp = inode->i_mtime.tv_sec > inode->i_ctime.tv_sec ?
inode->i_mtime.tv_sec : inode->i_ctime.tv_sec;
    perms = 0;
    perms |= (inode->i_mode & (S_IRUSR | S_IRGRP | S_IROTH)) ? 4
: 0;
    perms |= (inode->i_mode & (S_IWUSR | S_IWGRP | S_IWOTH)) ? 2
: 0;
    perms |= (inode->i_mode & (S_IXUSR | S_IXGRP | S_IXOTH)) ? 1
: 0;
    printk(KERN_INFO "sfs: sfs_write_inode (i_ino = %ld): %d
bytes @ %d secs w/ %o\n",
        inode->i_ino, size, timestamp, perms);

    return sfs_update(info, inode->i_ino, &size, &timestamp,
&perms);
}

int sfs_update(sfs_info_t *info, int vfs_ino, int *size, int
```

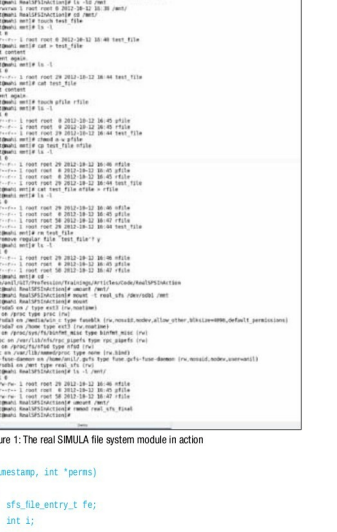

Figure 1: The real SIMULA file system module in action

```
*timestamp, int *perms)
{
    sfs_file_entry_t fe;
    int i;

    if (sfs_get_file_entry(info, vfs_ino, &fe) == -1)
    {
        return -1;
    }
    if (size) fe.size = *size;
    if (timestamp) fe.timestamp = *timestamp;
    if (perms && (*perms <= 07)) fe.perms = *perms;

    for (i = (fe.size + info->sb.block_size - 1) / info->sb.
block_size; i < SIMULA_FS_DATA_BLOCK_CNT; i++)
    {
        if (fe.blocks[i])
        {
            sfs_put_data_block(info, fe.blocks[i]);
            fe.blocks[i] = 0;
        }
    }

    return write_to_real_sfs(info, info->sb.entry_table_block_
```